github.com/mg-i-roc-maz/corsi · microservizi/lezione-8

LEZIONE 8 DI 8 · 3 ORE

Pattern avanzati e progetto finale

Messaggi che falliscono, che arrivano due volte, che non partono mai. Poi il negozio completo, e le vostre demo.

Le tre ore di oggi

Teoria · 45' | **Laboratorio · 1h45** | **Demo · 30'**

Teoria

Dead Letter Queue, consumer idempotente, Outbox, Saga orchestrata e coreografata, cenni di osservabilità.

Laboratorio

Tentativi e DLQ in notifications, doppioni riconosciuti, Outbox in orders, il giro completo del sistema.

Demo

Ogni gruppo: il suo negozio, un'estensione, un guasto dal vivo, una scelta. 5 minuti.

DA DOVE RIPARTIAMO

Le tre domande della lezione 7, e le tre risposte di oggi.

FALLISCE SEMPRE

Torna in coda all'infinito.

→ **tentativi contati e DLQ**

ARRIVA DUE VOLTE

Due email per un ordine.

→ **consumer idempotente**

NON PARTE MAI

Ordine salvato, evento perso.

→ **Outbox**

Il messaggio che fallisce sempre



Causa	Riprovare serve?
server di posta giù per 10 secondi	sì, **un po' dopo**
server di posta giù per un'ora	sì, ma non mille volte al secondo
un campo sbagliato nell'evento, un bug	**no**: fallirà identico ogni volta

Tre code: lavoro, attesa, parcheggio



Il contatore viaggia con il messaggio: header **x-attempt**. Alla terza volta, la DLQ.

Lo dice la coda, non il codice

```
await ch.assertQueue("notifications.order-created", {
  durable: true,
  deadLetterExchange: "shop.dlx",
  deadLetterRoutingKey: "notifications.order-created",
});
await ch.assertQueue("notifications.order-created.retry", {
  durable: true,
  messageTtl: 5000,
  deadLetterExchange: "",
  deadLetterRoutingKey: "notifications.order-created",
});
```

Un **nack senza requeue** sulla coda di lavoro: RabbitMQ lo manda a shop.dlx.



Gli argomenti di una coda **non si cambiano**: PRECONDITION_FAILED. Si cancella e si ricrea.

Il consumer idempotente

```
const first = await redis.set(
  `notifications:seen:${event.id}`, now,
  { NX: true, EX: 7 * 24 * 3600 });

if (first !== "OK") return; // doppione

try {
  await mailer.send(email);
} catch (err) {
  await redis.del(key); // riprova possibile
  throw err;
}
```

SET NX: scrive solo se la chiave non c'è, in un'operazione sola. Il primo vince, il doppione trova la chiave.

Si «prenota» l'evento **prima** dell'email: due copie di notifications non possono mandarla tutte e due.

Le scelte dietro tre righe

Situazione	Cosa fa notifications	Perché
evento già visto	ack, nessuna email	duplicates +1
email fallita	libera l'id, poi errore	il tentativo dopo la manda
Redis giù	manda **lo stesso**	meglio doppia che nessuna
evento visto 8 giorni fa	la manda di nuovo	la chiave è scaduta

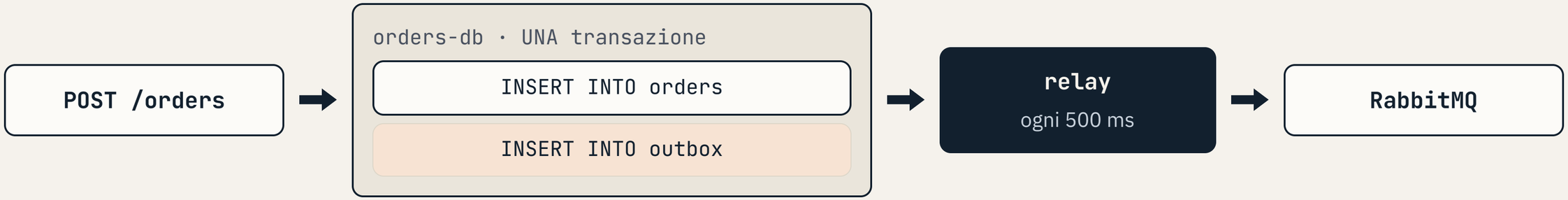
Per un'email «meglio doppia che nessuna». Per un **pagamento** la scelta sarebbe opposta: senza controllo, si aspetta.

OUTBOX

Due scritture, due posti, nessuna transazione comune.

orders salva l'ordine nel suo database e pubblica su RabbitMQ. Se cade in mezzo, o RabbitMQ è giù, l'ordine c'è e l'evento **no**.

L'evento si salva con l'ordine



RabbitMQ giù	orders cade dopo il COMMIT	Il relay cade dopo publish
l'evento aspetta nell'outbox	il relay lo trova al riavvio	lo ripubblica: **doppione**

La transazione

```
const client = await pool.connect();
await client.query("BEGIN");
const { rows } = await client.query(
  "INSERT INTO orders (customer_email, lines, total_cents) VALUES ($1, $2, $3) RETURNING ...", [...]);
await client.query(
  "INSERT INTO outbox (event_id, routing_key, event) VALUES ($1, $2, $3)",
  [event.id, "order.created", JSON.stringify(event)]);
await client.query("COMMIT"); // tutti e due, o (ROLLBACK) nessuno
```

Il relay: **SELECT … WHERE published_at IS NULL ORDER BY id**, pubblica, **UPDATE … SET published_at = now()**.

Altri modi di leggere l'outbox

Modo	Come	Quando
Polling (oggi)	un ciclo legge la tabella ogni N ms	semplice, basta per molti casi
Change Data Capture	uno strumento legge il log del database (WAL) e pubblica	tanti eventi, latenza bassa
LISTEN / NOTIFY	PostgreSQL sveglia il relay a ogni insert	meno attesa, stesso codice

Il principio non cambia: **una** scrittura sicura, poi la pubblicazione ripetibile.

Saga: una transazione che attraversa i servizi



Il passo 1 è già fatto, in un altro database. Non c'è un **ROLLBACK** che lo annulli.

Si **compensa**: un'azione nuova che annulla l'effetto della prima. Prenotare → restituire. Addebitare → rimborsare.

Orchestrata o coreografata?

Orchestrata

Un servizio **guida**: manda comandi e aspetta risposte.

```
orders → ReserveStock
orders → ChargeCard
  ← PaymentFailed
orders → ReleaseStock
```

Lo stato è in un posto solo. Più accoppiata.

Coreografata

Nessuno guida: ognuno **reagisce** agli eventi degli altri.

```
OrderPlaced → catalog
StockReserved → payments
PaymentFailed → catalog, orders
```

Disaccoppiata. Lo stato è sparso.

Ogni passo, la sua compensazione

Passo	Compensazione	Attenzione
prenotare le scorte	restituire le scorte	idempotente: due volte = una
addebitare la carta	rimborsare	il cliente vede due movimenti
mandare un'email	**un'altra email**	quella di prima non si cancella
confermare l'ordine	annullarlo (extra 4)	chi ha già letto OrderCreated?

Una compensazione non torna indietro nel tempo: **aggiunge** un fatto nuovo. E deve poter fallire e riprovare.

Osservare un sistema a eventi

Per ogni servizio: RED

- **R**ate: richieste al secondo
- **E**rrors: quante falliscono
- **D**uration: p95, p99

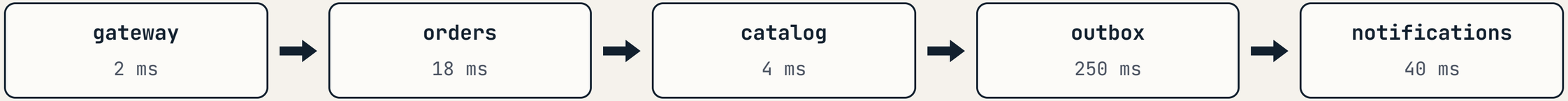
Per ogni coda

- messaggi **Ready**: cresce?
- età del più vecchio
- **DLQ** non vuota: allarme
- **outbox** pending: cresce?

Per ogni richiesta

- il **correlation ID** (lezione 4)
- anche dentro i messaggi
- OpenTelemetry: una traccia sola

Una traccia attraverso HTTP e code




```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
```

Lo stesso header W3C viaggia nelle chiamate HTTP **e** nelle proprietà dei messaggi: lo strumento ricostruisce la storia intera, con i tempi.

Il corso in una slide

Lezione	Problema	Risposta
1–2	un monolite che cresce	servizi separati, un container ciascuno
3	i servizi devono parlarsi	HTTP, un gateway davanti
4	un servizio lento blocca gli altri	timeout, retry, circuit breaker
5–6	il database è lento, tutti leggono le stesse cose	cache-aside, cache HTTP, invalidazione
7	aspettare tutti è fragile	eventi, code, consistenza eventuale
8	i messaggi falliscono, si ripetono, si perdono	DLQ, idempotenza, Outbox, Saga

Verifichiamo

1. Perché un messaggio illeggibile va subito nella DLQ, senza tentativi?
2. Perché il consumer «prenota» l'evento con SET NX prima di mandare l'email?
3. Con l'Outbox, un evento può essere pubblicato due volte? Quando?
4. Saga: che cos'è una compensazione, e perché non è un rollback?

Glossario

Termine	Significato
Poison message	Un messaggio che fallisce ogni volta allo stesso modo.
Dead Letter Queue	Dove finiscono i messaggi che non ce l'hanno fatta.
Consumer idempotente	Ricevere due volte lo stesso evento = riceverlo una volta.
Doppia scrittura	Salvare in due posti senza una transazione comune.
Outbox	L'evento si salva con i dati, un relay lo pubblica dopo.
Saga	Una sequenza di transazioni locali, ognuna con la sua compensazione.
Orchestrata, coreografata	Un servizio guida i passi; ognuno reagisce agli eventi.
RED	Rate, Errors, Duration: i tre numeri di ogni servizio.

Laboratorio

25'

1 · Tentativi e DLQ

TODO 1 e 2: notifications.

20'

2 · Idempotenza

TODO 3: i doppioni.

30'

3 · Outbox

TODO 4 e 5: orders.

25'

4 · Giro completo

Il controllo finale, poi la demo.

```
cd microservizi/lezione-8/start → docker compose up --build -d
```

Il repo della lezione

github.com/mg-i-roc-maz/corsi

```
cd corsi
git pull
cd microservizi/lezione-7/start
docker compose down -v
cd ../../lezione-8/start
docker compose up --build -d
```

Perché -v, questa volta

- la coda della lezione 7 non ha gli argomenti della DLQ, e non si possono aggiungere
- **orders-db** si crea solo a volume vuoto

Nuove dipendenze: **pg** in orders, **redis** in notifications.

Cosa c'è in start

```
services/notifications/src/
  broker.ts                    TODO 1: tre code
  consume.ts                   TODO 2: riprova o DLQ
  handlers.ts                  TODO 3: doppioni
  seen.ts                      SET NX in Redis
services/orders/
  db/init.sql, store-          ordini e outbox
  postgres.ts
  src/routes.ts                TODO 4
  src/outbox.ts                TODO 5: il relay
scripts/                       doppio, rotto, giro completo
```

È il negozio della lezione 7, con un database anche per orders.

Da leggere prima: **create()** in store-postgres.ts. È la transazione dell'Outbox.

1.1 · TODO 1 e 2: tre code, tentativi contati

TODO 1 · broker.ts

1. **shop.dlx** e la DLQ
2. coda di lavoro con **deadLetterExchange**
3. coda **.retry** con **messageTtl** che torna al lavoro

TODO 2 · consume.ts

Fallito, e...	Allora
tentativi rimasti	copia in .retry con x-attempt, poi ack
ultimo tentativo	nack senza requeue: DLQ

```
cd services/notifications && npm test # all'inizio: # fail 5
```

1.2 · Dove finisce un messaggio senza speranza

```
MAIL_FAIL_PERCENT=100 RETRY_DELAY_MS=3000 \
  docker compose up -d --build notifications
# un ordine, poi dopo 10 secondi:
curl localhost:8080/stats/notifications
# "received":3, "retried":2, "dead-lettered":1

./scripts/evento-rotto.sh
# messaggi nella DLQ: 2
```

Nella UI: il messaggio passa da **.retry** due volte, poi resta nella **.dlq**.

Il messaggio rotto: **nessun** tentativo, subito nella DLQ.

2 · TODO 3: il doppione riconosciuto

```
./scripts/evento-doppio.sh
# prima del TODO 3: due email a doppio@example.com

# TODO 3 in src/handlers.ts, poi:
docker compose up -d --build notifications
./scripts/evento-doppio.sh
# "emailsSent":1, "duplicates":1

docker compose exec redis redis-cli \
  KEYS 'notifications:seen:*'
```

Lo script pubblica **due volte** lo stesso evento, stesso id: è quello che fa una riconsegna.

Con un'email fallita, l'id va **liberato**: lo dice il test «email fallita: l'id si libera».

3.1 · TODO 4 e 5: l'Outbox

TODO 4 · routes.ts

A **store.create**, come terzo argomento, la funzione che costruisce l'evento:

```
(created) ⇒ [{
  routingKey: ORDER_CREATED,
  event: orderCreatedEvent(
    created, request.id) }]
```

TODO 5 · outbox.ts

1. **pendingEvents**(batch)
2. pubblica **in ordine**
3. al primo errore: **fermati**
4. **markPublished** dei riusciti

```
cd services/orders && npm test # all'inizio: # fail 5
```

3.2 · RabbitMQ giù, nessun evento perso

```
docker compose up -d --build orders
docker compose stop rabbitmq
./scripts/ordini.sh 3 # tre volte 201
curl localhost:8080/stats/outbox
# {"pending":3,"published":0}
docker compose start rabbitmq
# dopo qualche secondo:
# {"pending":0,"published":3} e tre email
```

```
docker compose exec orders-db \
  psql -U shop -d orders -c \
  "SELECT id, routing_key,
   published_at FROM outbox"
```

Nella lezione 7 questi tre eventi sarebbero persi.

4 · Il giro completo

```
./scripts/giro-completo.sh
OK catalog è pronto
OK orders è pronto
OK notifications è pronto
OK seconda lettura di p-1: HIT nel gateway
OK ordine creato (HTTP 201)
OK outbox vuoto: l'evento è partito
OK email di conferma inviata a giro…

7 OK, 0 NO
```

Gateway, cache, catalog, orders, outbox, RabbitMQ, notifications, Redis: **tutto il corso** in sette righe.

Poi: la demo del gruppo, PROGETTO.md.

Checkpoint

Verifica	Esito atteso
`npm test in orders e notifications`	tutto verde, alcuni saltati
`posta sempre giù, un ordine`	3 tentativi, poi nella DLQ
`./scripts/evento-rotto.sh`	subito nella DLQ
`./scripts/evento-doppio.sh`	una email, duplicates 1
`RabbitMQ fermo, 3 ordini`	pending 3, poi 0 e tre email
`./scripts/giro-completo.sh`	7 OK, 0 NO

Se qualcosa non va

Sintomo	Causa e rimedio
`PRECONDITION_FAILED nei log`	Coda della lezione 7: Delete dalla UI, o down -v
`Le copie si moltiplicano`	TODO 2: manca l'ack dopo sendToQueue
`Nella DLQ al primo errore`	TODO 2: confronto con maxAttempts sbagliato
`Mai un'email dopo un fallimento`	TODO 3: l'id non viene liberato
`Nessuna email, outbox vuoto`	TODO 4: l'evento non è scritto con l'ordine
`outbox pending che non scende`	TODO 5, o RabbitMQ giù: log di orders
`orders unhealthy`	orders-db senza tabelle: down -v

GLI ULTIMI 30 MINUTI

Le vostre demo.

1'

Il sistema

docker compose ps e il giro completo.

2'

L'estensione

Cosa avete aggiunto, dal vivo, con i test.

1'

Un guasto

Fermate un pezzo: cosa regge, cosa no.

1'

Una scelta

Cosa avete deciso, cosa avete scartato.

Le domande da aspettarsi

- Se questo servizio si spegne, cosa smette di funzionare? Cosa invece continua?
- Questo messaggio può arrivare due volte? Cosa succede, se succede?
- Questo dato può essere vecchio? Di quanto, al massimo?
- Dove guardereste per capire un problema in produzione?

FINE DEL CORSO

Da un solo servizio a un sistema che regge.

Gateway, cache a tre livelli, timeout e circuiti, eventi che non si perdono e non si ripetono. Tutto con un solo docker compose up.

`github.com/mg-i-roc-maz/corsi · microservizi/`

PER CHI FINISCE PRIMA, PER CASA, O PER IL PROGETTO

Esercizi extra

#	Esercizio	Livello	Dove
1	Chi c'è nella DLQ?	base	Management UI
2	Rimettere in coda la DLQ	medio	notifications
3	Due copie di orders	avanzato	orders
4	Annullare un ordine	medio	orders, notifications
5	La Saga del pagamento	carta e penna	—

Testi in lezione-8/extra/README.md. Per 2, 3 e 4 ci sono test pronti; le soluzioni sono in lezione-8/extra/soluzione.

Chi c'è nella DLQ?

Nessuna riga di codice: la Management UI.

1. un messaggio nella DLQ (posta giù, o evento-rotto.sh)
2. Queues › **.dlq** › Get messages
3. leggi l'header **x-death**: coda, motivo, quante volte
4. confronta: tre tentativi o illeggibile

Domanda

Chi dovrebbe guardare la DLQ, e quando? Cosa dovrebbe far scattare un allarme?

Rimettere in coda la DLQ

Il server di posta è tornato: i messaggi della DLQ vanno rimandati.

- **replayDeadLetters**: channel.get uno alla volta
- via **x-attempt** e **x-death**, nella coda di lavoro, ack
- gli illeggibili restano dove sono

```
docker compose exec notifications \
  node dist/replay-cli.js
```

Domanda

Rimettere in coda un messaggio già gestito manda un'email doppia?

Due copie di orders

Due relay leggono gli stessi eventi: ogni evento parte **due volte**.

```
SELECT ... FROM outbox
WHERE published_at IS NULL
ORDER BY id LIMIT 50
FOR UPDATE SKIP LOCKED
```

Ogni relay blocca le righe che prende; l'altro le salta.

Prova

--scale orders=2, venti ordini, e guarda **duplicates** in /stats/notifications, prima e dopo.

Annullare un ordine

POST /orders/:id/cancel: stato «cancelled» e un evento **OrderCancelled** nell'outbox, insieme.

- due volte: **409**, nessun secondo evento
- notifications: binding **order.cancelled** e l'email

Domanda

Le scorte prenotate: le restituisce orders chiamando catalog, o catalog ascoltando l'evento?

La Saga del pagamento

Tre passi: **prenotare** (catalog), **addebitare** (payments), **confermare** (orders). Se l'addebito fallisce, le scorte tornano.

1. disegna la versione **orchestrata**
2. disegna la versione **coreografata**
3. la compensazione di ogni passo

Infine

In quale delle due è più facile rispondere: «a che punto è l'ordine o-7?»

Per approfondire

C. Richardson · *Microservices Patterns*	Manning, 2018: Saga, Outbox, consumer idempotente, con esempi
microservices.io · *Transactional outbox, Saga*	le schede dei pattern, una pagina ciascuna
RabbitMQ · *Dead Letter Exchanges*	rabbitmq.com/docs: argomenti, x-death, motivi
H. Garcia-Molina, K. Salem · *Sagas*	SIGMOD 1987: l'articolo che ha dato il nome
OpenTelemetry · *JavaScript*	opentelemetry.io: tracce automatiche per Node, HTTP e amqplib
Debezium · documentazione	debezium.io: Change Data Capture e outbox event router